и немножко GitHub

Часть вторая

Обо мне

Никита Бездольный, front-end

Написал первое приложение в 2003 году(на Pascal), с тех пор не отпускает

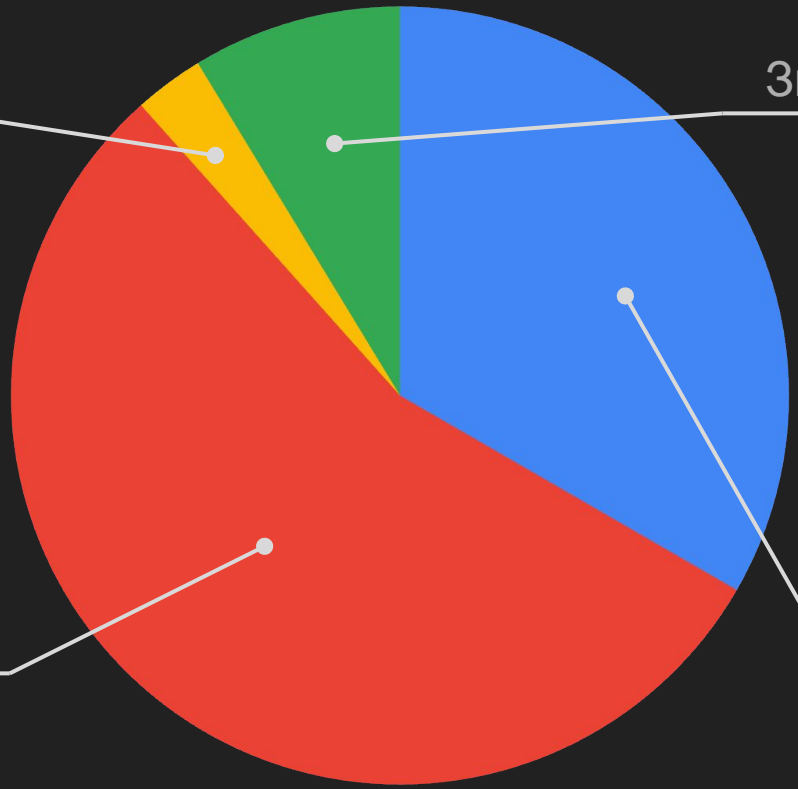
На GitHub с 2013 года

Много писал на PHP, привет поклонникам Yii2

Выпускник Школы программистов hh 17-18

Сейчас Frontend-разработчик команды A-Team

О вас


Что такое git?
2.9% (+2.9)
Знаю, но не работал
8.7% (-17.7)
Небольшой опыт
55.1% (+26.8)
Широкий опыт
33.3% (-12)

Краткое содержание предыдущей части

Научились создавать локальный репозиторий и клонировать удалённый

Узнали, что такое индекс, рабочая копия, коммит, ветка, тэг

Узнали базовые команды для работы с git (init, clone, checkout, switch, restore, add, commit, merge и несколько других)

Узнали что у этой части будет домашнее задание

Цели и планы

Конфликты и их решение

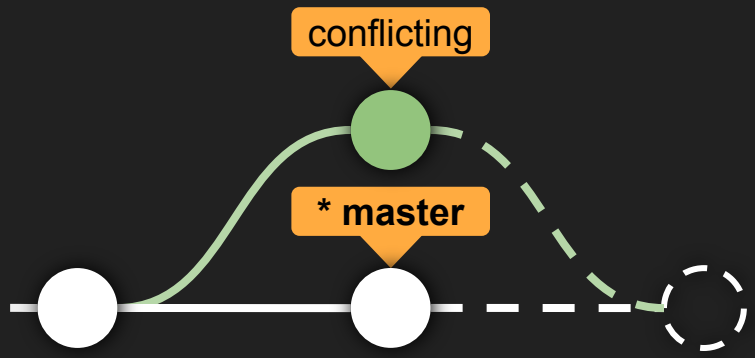
Работа с историей коммитов в git

- revert, reset, rebase [--onto], интерактивный rebase
- Удаление объектов git

Работа с GitHub

- pull, push
- Pull requests, работа с удалёнными репозиториями

Простое начало



Тривиальная ситуация, нужно влить изменения ветки conflicting в ветку master, создав merge commit

Напишите в чат команду, которая это сделает

Сложный путь

```
$ git merge conflicting
Auto-merging sample.md
CONFLICT (content): Merge conflict in sample.md
Automatic merge failed; fix conflicts and then commit the result.
$
```

Иногда попытка влить изменения может закончиться конфликтом

К конфликтам могут привести разные команды, например: merge, pull, rebase, stash pop, revert, cherry-pick

Причина их возникновения всегда одинаковая

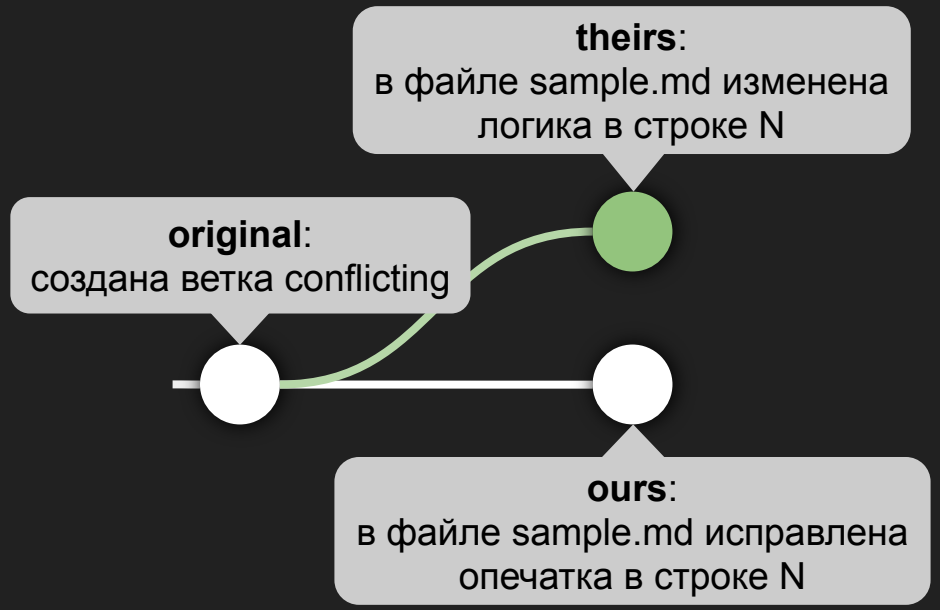
Перекрёсток трёх дорог

На самом деле, при слиянии, двух веток, git объединяет три версии файла. При вливании ветки conflicting в ветку master, будут использоваться:

1. Изначальная версия (та, которая была в момент создания ветки conflicting), называется original
2. Текущая версия ветки master, называется ours или yours
3. Текущая версия ветки conflicting, называется theirs

Это называется 3-way-merge, он используется всегда, до этого мы рассматривали ситуации, когда одна из текущих версий совпадала с изначальной.

Что произошло



При попытке слияния git видит, что обе текущие версии файла не совпадают с изначальной, и не может определить, какая из них важнее

Причина всех конфликтов

Мы пытаемся внести изменения в строки файла, которые на текущий момент уже изменены:

1. Мы взяли ветку и отредактировали файл
2. Кто-то сделал то же самое (фиксил баг в том же файле)
3. Чужие изменения были влиты в master
4. Мы пытаемся влить изменения в master
5. Git видит, что мы изменили исходный файл, а не текущий, значит в наших изменениях не учтены текущие
6. Конфликт

Остановись, мгновенье

В случае возникновения конфликта – производимая операция будет остановлена

Большинство операций можно запустить с параметром --abort для отмены и возврата к предыдущему состоянию

После разрешения конфликта revert, rebase и cherry-pick можно продолжить, запустив с параметром --continue

stash pop просто не сработает и завершится без применения изменений, они так и останутся в стэше

Ёлки, палки!

```
$ cat sample.md
...
<<<<<<< HEAD
Это исправленное содержимое ветки master
=======
Это текущее содержимое ветки conflicting
>>>>>>> conflicting
...
$
```

В случае конфликта git добавит в файл метки, определяющие границы конфликта

Внутри меток видно разное содержимое одних и тех же строк конфликтующих версий

Верхняя часть – это ours (master), нижняя – theirs (conflicting).

И что делать?

Чтобы решить конфликт руками, нужно отредактировать конфликтующий файл и оставить тот вариант, который нужен, убрав метки и всё лишнее

В некоторых случаях решение конфликта будет выходить за метки, которые добавляет git – это нормально

Можно выбрать какой-то третий вариант, просто написав его вместо двух предложенных

После редактирования, добавляем в индекс и коммитим, как обычно

Разрешение конфликта

`$ nano sample.md`	Редактируем файл текстовым редактором
`$ git diff`	Смотрим, что получилось

`...`
`- Исправленное содержимое строки из ветки master`
`-Содержимое строки из ветки conflicting`
`++Исправленное содержимое строки из ветки conflicting`
`...`	

`$ git add sample.md`	Добавляем в индекс
`$ git commit`	Коммитим, будет запущен текстовый редактор, в котором можно изменить сообщение коммита
`[master 99bf0c4] Merge branch 'conflicting'`	
`$`	

git diff – показывает различия между версиями файлов

Без параметров показывает различия между рабочей копией и индексом

Давайте покажу

Если вы смотрите эту презентацию
не на лекции, то смотрите запись

Как-то это кустарно

Есть множество инструментов, позволяющих более наглядно и удобно показать границы конфликта.

У git для их запуска есть специальная команда mergetool. Подробнее.

Запускает один из инструментов, поддерживающий обработку меток конфликтов git, потом забирает результат.

Самих таких инструментов полно, выбирать можно по своему вкусу из списка, который git пишет при первом запуске команды.

RE
vert
set
base

Операции с историей коммитов

Revert

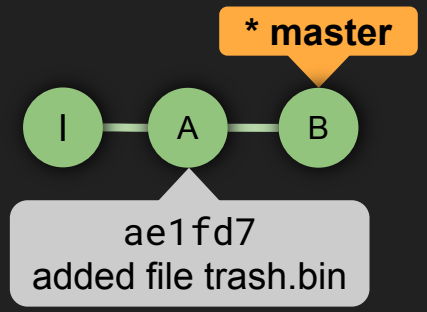
git revert <commit>... – позволяет откатить изменения путём создания нового коммита с обратными изменениями

Самая безобидная из трёх, не изменяет уже существующую историю

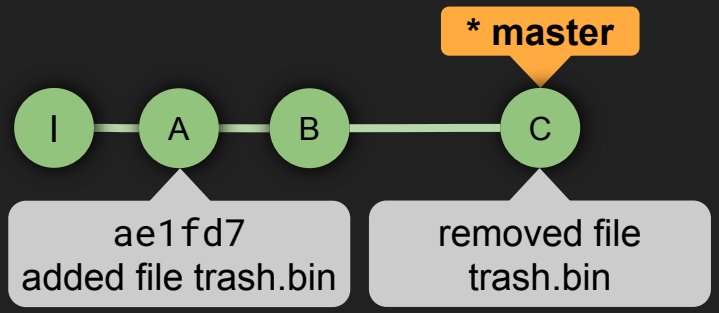
Полезно, когда откатываемые изменения уже давно являются частью основной ветки и удалять их из истории нецелесообразно

Если можно обойтись без revert – лучше обойтись, история будет чище

Схема revert




```
$ git
revert
ae1fd7
```



История коммитов остаётся неизменной, новый коммит добавляется в конец

Эта операция может вызывать конфликты

Для отката нескольких коммитов – их нужно перечислить в команде в обратном порядке. Почему?

Ссылки в git

Когда в любой команде вам нужно указать коммит, у вас есть несколько вариантов:

- полный хэш коммита
- сокращённый хэш коммита, сокращать можно до любого количества символов, пока не случится коллизия имён
- имя любого указателя на коммит (ветки, тэга)
- имя любого указателя удалённого репозитория
- Использовать HEAD

С любым из этих вариантом можно использовать два спецсимвола ~ и ^

Reset

git reset <link> – перемещает указатель текущей ветки на другой коммит

Если после перемещения указателя – на изначальный коммит не осталось ссылок – он считается удалённым. Физически из репозитория он будет удалён не менее, чем через 30 дней и его можно восстановить.

Имеет три режима, указываемых параметром

reset --hard

Рабочая копия и индекс берутся из назначения

Файлы будут заменены взятыми из коммита назначения

Эта команда гарантированно удаляет все изменения, лучше быть внимательнее, когда её выполняете, потому что можно удалить лишнее.

reset --soft

Рабочая копия и индекс берутся из источника

Файлы остаются нетронутыми и попадают в индекс

Если сделать `git reset --soft HEAD~1 && git commit` – по сути, мы просто ещё раз создадим последний коммит. Это будет новый коммит с новым хэшем, но одинаковым содержимым.

reset --mixed

Рабочая копия берётся из источника, индекс – из назначения. Это поведение по умолчанию

Файлы остаются нетронутыми, но в индекс не попадают

Полезно, когда накоммитили лишнего или хотите разбить последний коммит на несколько

Схема reset

Перемещать указатель можно куда угодно, на другую ветку, на коммит, на тэг

Изменяет историю коммитов

Если пересоздать коммит B – у него будет другой хэш

Rebase

git rebase <link> – переносит все коммиты текущей ветки так, чтобы она начиналась из коммита <link>

Полезно, когда в master появляются изменения, которые могут повлиять на вашу ветку, или от которых может зависеть ваш код

Или вы хотите изменить историю своих коммитов удобным способом (почистить лишнее, поменять что-то местами, переименовать, объединить или разделить коммиты)

Rebase vs Merge

В файлах результаты работы rebase идентичны результатам merge

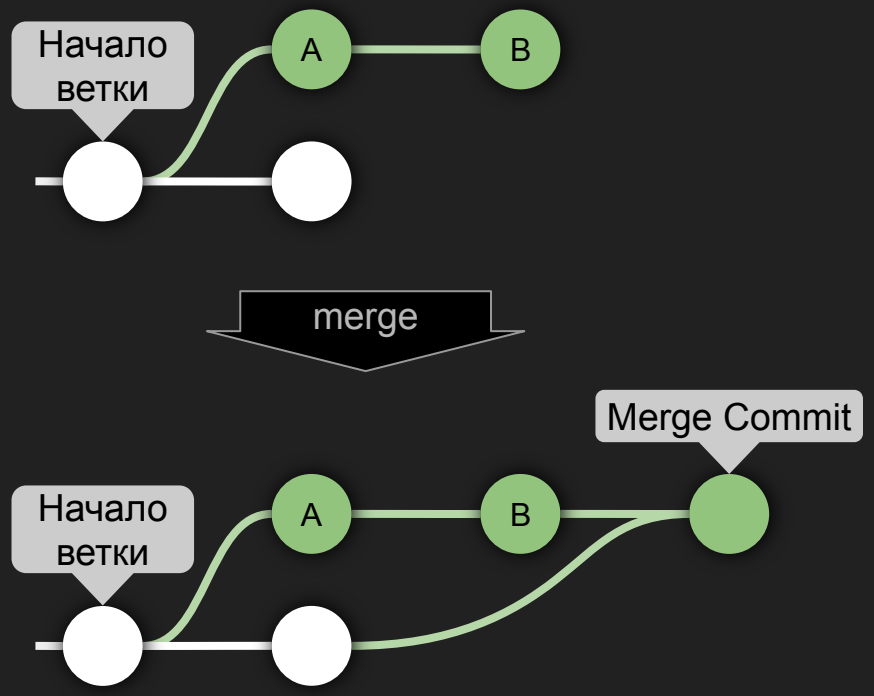
При rebase тоже могут быть конфликты

Принцип, по которому применяются изменения абсолютно разный

Сам по себе rebase не создаёт новых коммитов

У rebase есть интерактивный режим, с помощью которого, можно полностью переписать историю коммитов

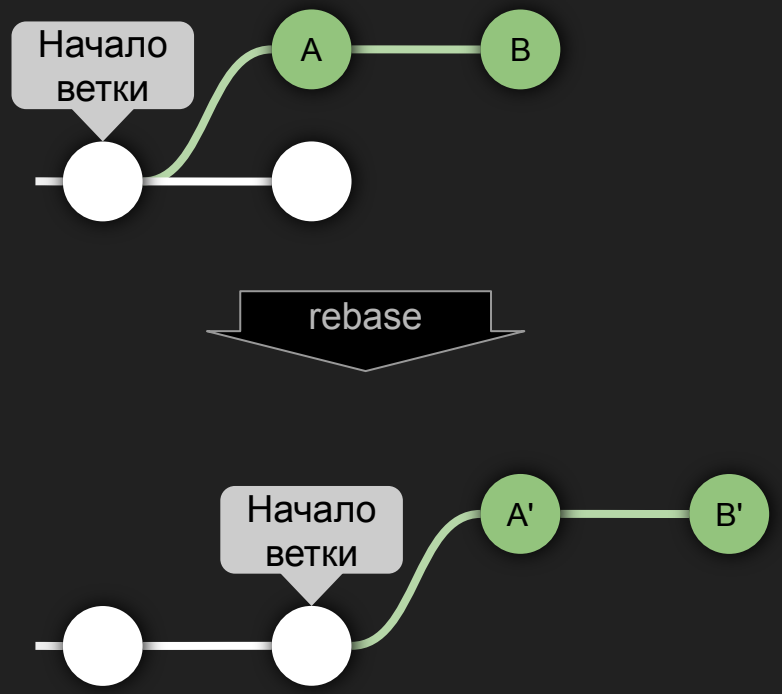
Схема Merge



Если вы хотите, чтобы в вашей ветке оказались изменения master

Fast-forward сделать невозможно, потому что есть коммиты в базовой ветке

Схема Rebase



Если вы хотите, чтобы в вашей ветке оказались изменения master

Название отражает суть, мы перебазируемся

Коммиты ветки применяются последовательно

Хэши коммитов будут другими

Rebase -i

Интерактивный ребейз нескольких коммитов, для каждого из них позволяет:

- Удалить коммит (drop)
- Изменить коммит (edit)
- Изменить сообщение коммита (reword)
- Поменять местами с другим (прям руками строки)
- Объединить с другим (squash, fixup)
- Запустить консольную команду на рабочей копии коммита (exec)
- Ничего не делать с коммитом (pick, по умолчанию)

Rebase останавливается при edit или конфликтах. После завершения операции, можно продолжить rebase, выполнив git rebase --continue

Например

Фу, какая гадость!

```
$ git log -6 --oneline
e259ac2 (HEAD -> rebase-me) Fix
14e8c14 Stop using alert and use console
69db971 THIS MUST BE FIRST COMMIT
80f3b03 Empty commit
3d561f2 asdhdf
a1530e4 Add project files
$
```

git log этой ветки выглядит страшненько, кто-то коммитил в спешке, и получилась очень корявая история

Красивая и понятная история коммитов в репозитории – это помощь другим разработчикам, и будущему себе

Сделать конфетку

```
$ git rebase -i HEAD~6
```

git rebase -i <link> – запуск интерактивного rebase. Параметр <link> определяет начальный коммит

Это запустит текстовый редактор, в котором есть подробная справка о том, как использовать интерактивный режим.

HEAD~6 – относительная ссылка на коммит, в данном случае – шестой коммит в истории относительно текущего

Надо что-то менять

Давайте для asdhdf и Fix сделаем fixup

Удалим пустой коммит

Перенесем THIS MUST BE FIRST COMMIT в начало, и сделаем ему reword в 'Add npm package file'

TERRIBLETERRIBLEDAMAGE

Starcraft

Правильная инструкция для rebase -i:

```
reword 69db971 THIS MUST BE FIRST COMMIT
pick a1530e4 Add project files
fixup 3d561f2 asdhdf
pick 14e8c14 Stop using alert and use console
fixup e259ac2 Fix
```

После сохранения и выхода из редактора, он будет запущен еще раз для переименования коммита – вводим новое название, сохраняем, выходим.

```
[detached HEAD 59af7aa] Add npm package file
 1 file changed, 1 insertion(+)
 create mode 100644 package.json
Successfully rebased and updated refs/heads/rebase-me.
```

О дивный новый log

```
$ git log -3 --oneline
6e22746 (HEAD -> rebase-me) Stop using alert and use console
a77559a Add project files
292750c Add npm package file
$
```

Стало понятнее?

Всё меняется

```
$ git log -6 --oneline
e259ac2 (HEAD -> rebase-me) Fix
14e8c14 Stop using alert and use console
69db971 THIS MUST BE FIRST COMMIT
80f3b03 Empty commit
3d561f2 asdhdf
a1530e4 Add project files
$
```

```
$ git log -3 --oneline
6e22746 (HEAD -> rebase-me) Stop using alert and use console
a77559a Add project files
292750c Add npm package file
$
```

Заметили, что у всех коммитов поменялись хэши?

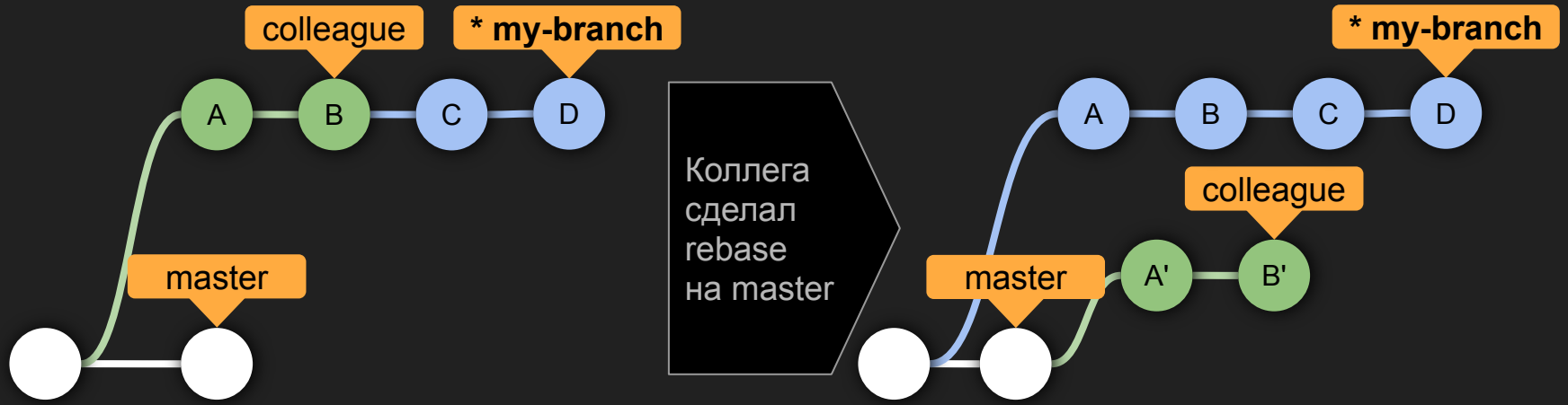
Ветка теперь совершенно другая, если из одного из старых коммитов создавалась другая ветка – в ней не будет ни одного из новых коммитов

Есть вопросы?

Нужно демо?

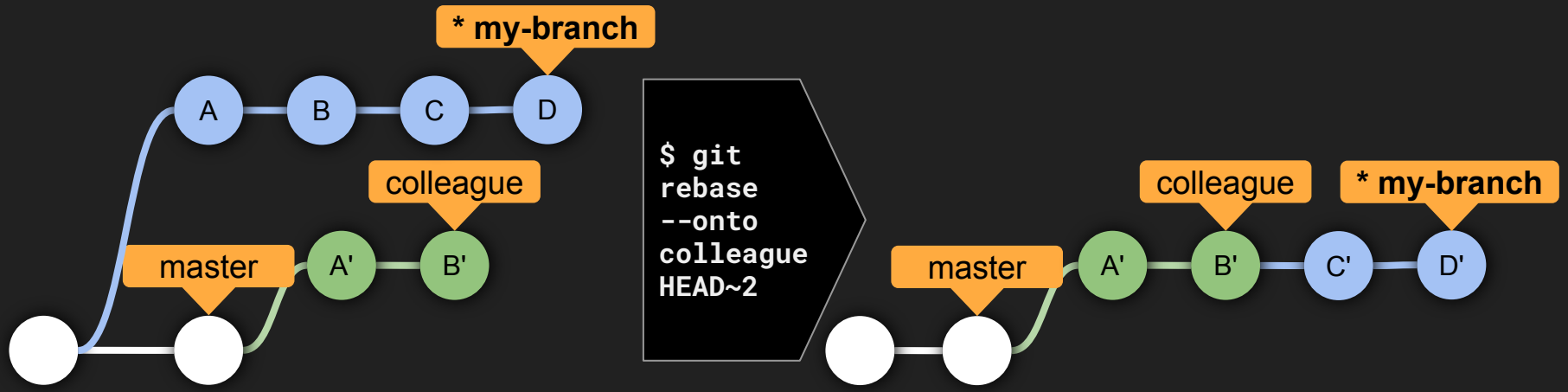
Rebase --onto

Иногда, при коллективной работе, ваша ветка начинается не из master, а из ветки вашего коллеги. Если коллега отребейзит свою ветку, то ваша останется со старыми коммитами, и при простом ребейзе, заберёт их все с собой, потому как теперь эти коммиты – её часть



Rebase --onto

Можно перенести часть коммитов с одной ветки на другую, с помощью
git rebase --onto <TARGET> <FROM> [<BRANCH>],
где FROM – это первый коммит, который нужно взять. Все, которые были до него, потеряются



Не самая полезная команда

git cherry-pick <commit> – позволяет скопировать один или несколько коммитов и применить их к текущей рабочей копии

Ситуативно полезна, если нужно надёргать по одному коммиту из кучи веток. Иногда заменяет ребейз или мердж, но без слияния веток

Опасна при неконтролируемом использовании и чередовании своих и созданных cherry-pick коммитов, может подпортить историю

Если видите, что без cherry-pick не обойтись – значит что-то не так с репозиторием, или моделью ветвления

Прервёмся?

Дальше будет полегче

REmote

Удалённые репозитории

Плюсы удалённого репозитория

Доступен из любого места с интернетом

Меньше риск потерять данные из-за поломки оборудования

Коллективная работа, ревью кода

Автоматическая сборка, тестирование, деплой

Релизы, пакеты, распространение

GitHub.com

Изначально – хранилище репозиториев, с возможностями git merge

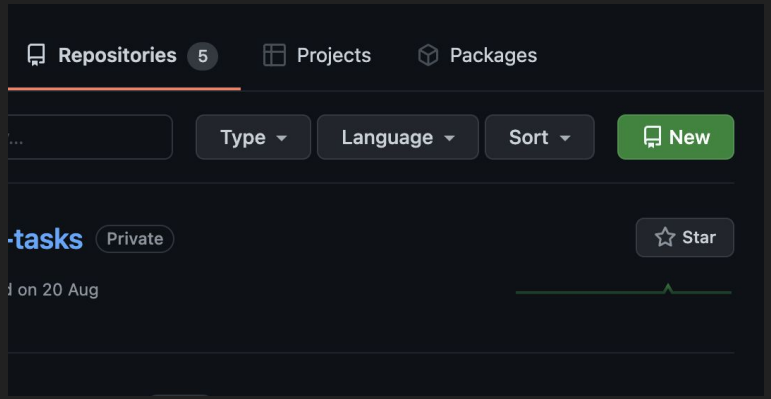
Сейчас – целый оркестр инфраструктуры, помогающей разработчикам

- отслеживание событий и реагирование на изменения
- пулл-реквесты и задачи
- комментарии и код-ревью
- веб-страницы из репозиториев и документация в wiki
- публичные и приватные репозитории
- agile и скрам-доски
- ...

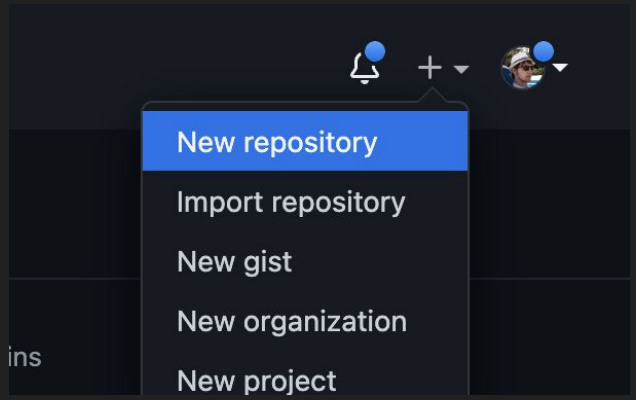
Показать им всем!

Вы можете перенести локальный репозиторий на GitHub, но сначала его нужно создать

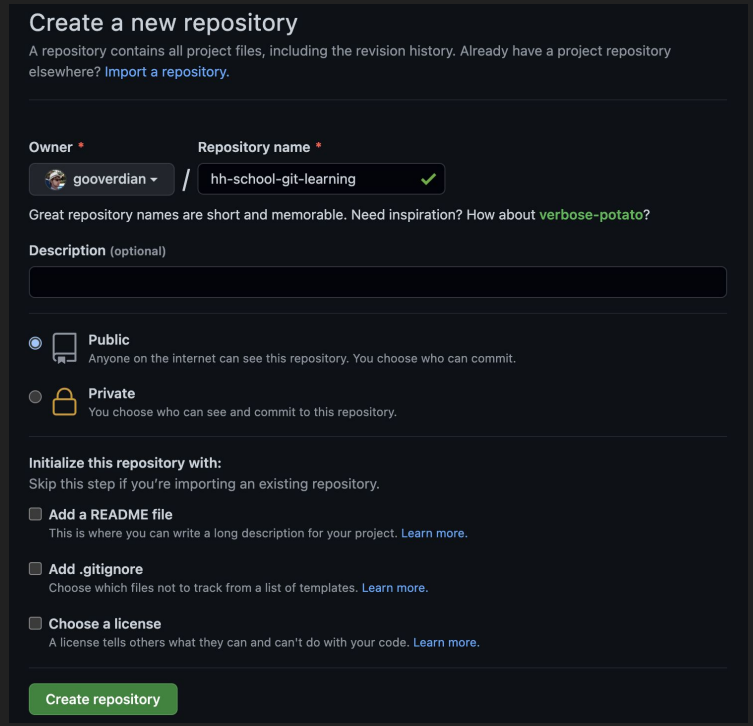
Чтобы это сделать, нужно зайти под своим аккаунтом и нажать New в меню репозиториев, или New repository в верхнем правом меню +



или



Создание репозитория



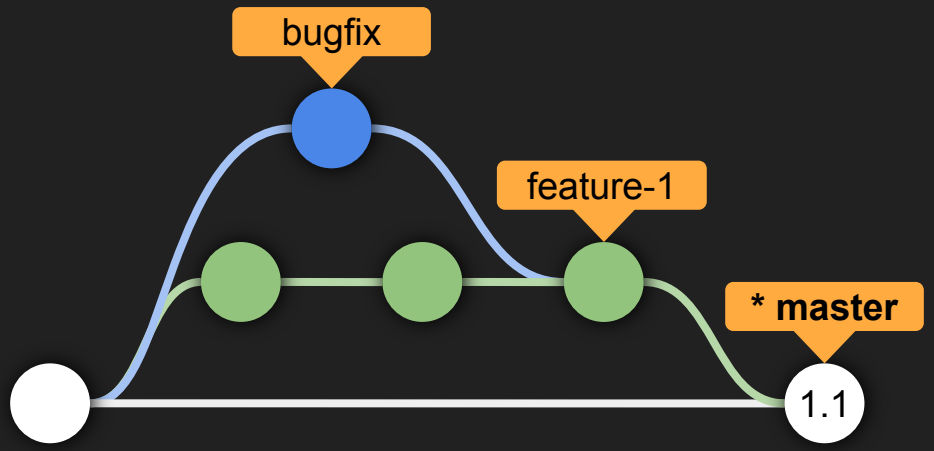
Имя – латиница, цифры и дефис

Описание – это не README, показывается в списке репозиториев

- публичный – это значит, что его увидят все пользователи GitHub, обновлять без вашего ведома никто не сможет
- приватный – вы сами управляете разрешениями

Если отметить галки – в репозиторий будут добавлены коммиты, если не отмечать – репозиторий будет пустым

Старый друг



Репозиторий Анатолия из предыдущей части

В далёком, далёком репозитории

```
$ git remote add origin https://github.com/<USERNAME>/<REPOSITORY_NAME>
$
```

git remote add <NAME> <LINK> – добавляет связь с удалённым репозиторием, у связи есть имя и ссылка

Если удалённый репозиторий один – используйте имя origin, оно почти везде по умолчанию, и к нему все привыкли

Ссылку на git репозиторий, можно взять со страницы репозитория на GitHub

С помощью git remote можно также изменять и удалять связи

Push me!

```
$ git push origin -u master feature-1 bugfix 1.1
Counting objects: 19, done.
Delta compression using up to 12 threads.
Compressing objects: 100% (14/14), done.
Writing objects: 100% (19/19), 2.50 KiB | 512.00 KiB/s, done.
Total 19 (delta 5), reused 0 (delta 0)
remote: Resolving deltas: 100% (5/5), done.
To https://github.com/gooverdian/hh-school-git-learning.git
 * [new branch]      master -> master
 * [new branch]      feature-1 -> feature-1
 * [new branch]      bugfix -> bugfix
 * [new tag]         1.1 -> 1.1
Branch 'master' set up to track remote branch 'master' from
'origin'.
Branch 'feature-1' set up to track remote branch 'feature-1'
from 'origin'.
Branch 'bugfix' set up to track remote branch 'bugfix' from
'origin'.
$
```

git push – отправляет объекты в удалённый репозиторий. Можно отправить ветки и теги, разделив их пробелом. Первый параметр – имя remote.

-u или --set-upstream-to – с этим флагом, все ветки, которые были отправлены станут отслеживаемыми. При следующих пушах этих веток, origin писать уже не нужно

push – единственная команда, которая меняет что-либо в удалённом репозитории

Набросим ветку на GitHub

Нужно создать новую ветку из текущего master, добавить в неё коммит, и сделать push origin -u <BRANCH>

GIVEBIOAMMO

BioShock

1	`$ git switch -c new-branch` `Switched to a new branch 'new-branch'`	Создаем ветку new-branch
2	`$ git log --oneline > git.log`	Создаём какой-нибудь контент для коммита
3	`$ git add git.log`	Добавляем в индекс

4	`$ git commit -m "Add current log"`
	`[new-branch c0b6553] Add current log`
	`1 file changed, 7 insertions(+)`
	`create mode 100644 git.log`
		Коммитим

5	`$ git push -u origin new-branch`
	`Counting objects: 3, done.`
	`Delta compression using up to 12 threads.`
	`Compressing objects: 100% (3/3), done.`
	`Writing objects: 100% (3/3), 503 bytes | 503.00 KiB/s, done.`
	`Total 3 (delta 0), reused 0 (delta 0)`
	`To github.com:gooverdian/hh-school-git-learning.git`
	`* [new branch] new-branch -> new-branch`
	`Branch 'new-branch' set up to track remote branch 'new-branch' from 'origin'.`
	`$`
		Отправляем ветку в удалённый репозиторий

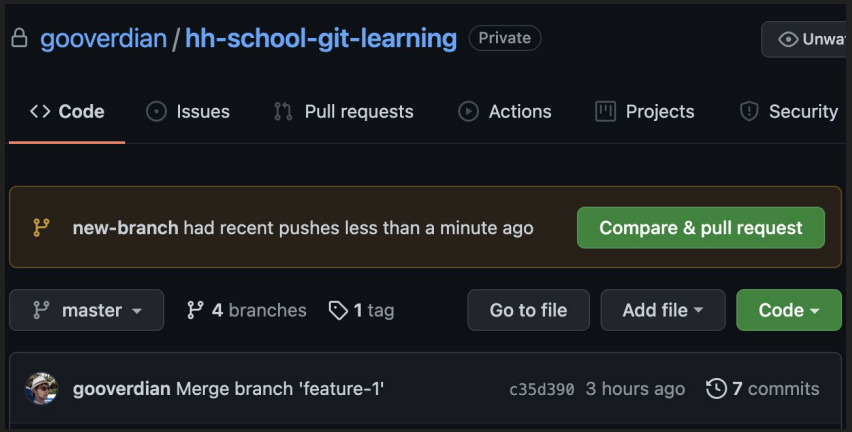
Обновляем репозиторий не используя git

Отправленная ветка появилась на GitHub, её видно в уведомлении, или можно выбрать в селекторе веток

GitHub позволяет проводить merge операции прямо на сайте, они называются Pull request

В чужих публичных репозиториях – это единственный способ сделать merge в master

На самом деле, GitHub позволяет и коммиты делать прямо на сайте.



Чтобы создать такой Pull request – для только что обновлённой ветки можно нажать Compare & pull request, для любой другой – "Pull requests" > "New pull request" и выбрать ее в окне

Обновляем репозиторий не используя git

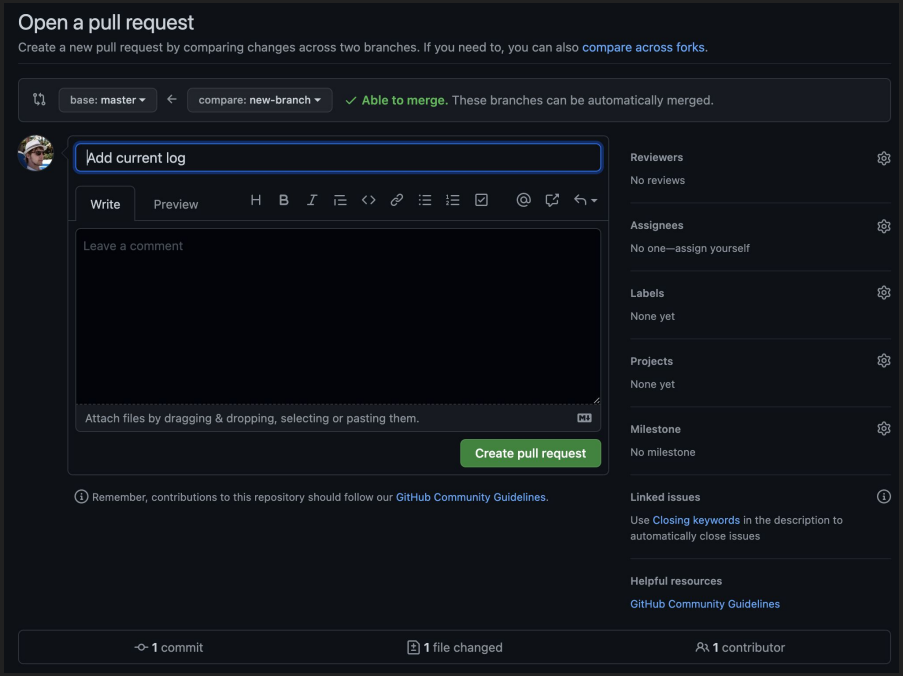
У Pull request есть базовая ветка, название и описание

Базовая ветка - та, в которую будут вливаться изменения

По умолчанию, название равно сообщению коммита, если он один, или облагороженному названию ветки, если коммитов несколько.

Описание поддерживает markdown

В нижней части можно увидеть все изменения, которые произойдут при слиянии



Code review

Если в репозитории практикуется ревью кода – после создания пул реквеста нужно выбрать ревьювера, обычно, GitHub предлагает ревьювера сам на основе истории коммитов

Ревью сводится к комментированию предлагаемого кода, исправлению ошибок и изменениям логики, иногда к закрытию пул реквеста

Практика хорошего ревью предполагает объективность и внимательность, нужно стараться комментировать решение без отношения к автору кода

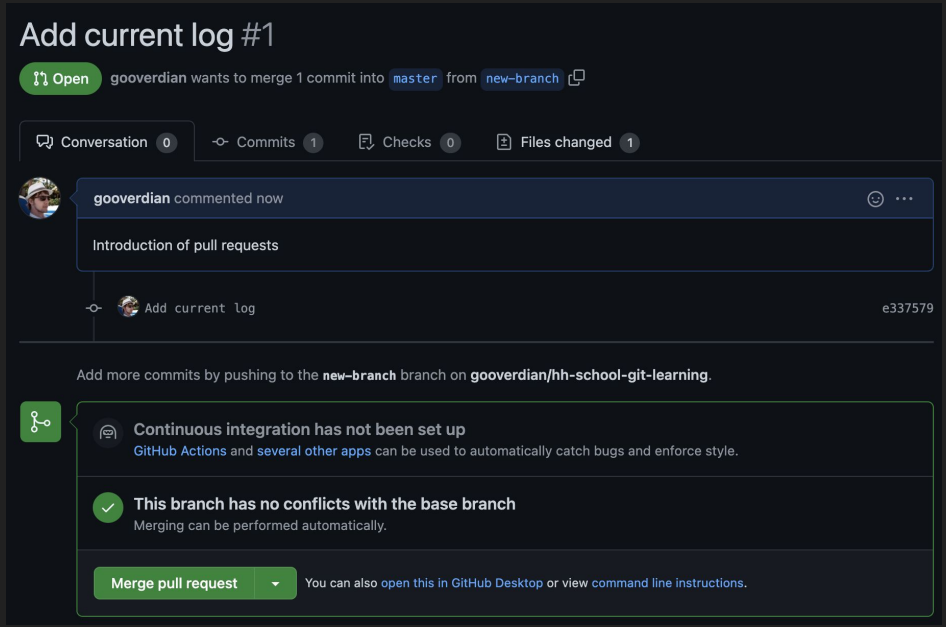
Почти все наши домашние задания, и весь проект так или иначе связаны с ревью кода, так что примеры будут по ходу обучения

Обновляем репозиторий не используя git

У каждого pull-request или issue есть уникальный номер, указан после хэштега, если написать #1 в комментарии, GitHub вставит ссылку

Во вкладках pull request можно посмотреть коммиты и изменённые файлы, оставить комментарии к отдельным строкам или ко всему коду

Обычно, в крупных проектах, кнопку merge никто не нажимает, это делают боты, когда убедятся, что всё хорошо



Знать бы ещё об этом

```
$ git fetch
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (1/1), done.
From github.com:gooverdian/hh-school-git-learning
   a2f207f..bda60a6  master     -> origin/master
$
```

git fetch – синхронизация изменений удалённого репозитория

Обновляет информацию в локальном репозитории, не трогает ветки, индекс и рабочую копию

Позволяет узнать, что произошло в удалённом репозитории

Get over here!

```
$ git switch master
Switched to branch 'master'
Your branch is behind 'origin/master' by 2 commits, and can be fast-forwarded.
  (use "git pull" to update your local branch)
$ git pull
Updating a2f207f..bda60a6
Fast-forward
 new-file.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 new-file.txt
$
```

git pull – получение ветки из удалённого репозитория

Делает fetch, а потом ещё и обновляет ветки, индекс и рабочую копию

Почему пришло на один коммит больше, чем мы добавили в ветку?

Полагается пошалить

Все объекты репозитория – ветки, тэги, коммиты – можно удалять

Так как ветки и тэги – это просто указатели, удаляются они просто:
git branch -d <NAME>
git tag -d <NAME>

Удалить коммит не так просто, все коммиты, которые были после него, будут пересчитаны и будет построено новое дерево.

Семь бед – один reset

```
$ git reset --hard 1.1
HEAD is now at a2f207f Merge branch 'feature-1'
$ git push
...
error: failed to push some refs to 'git@github.com:gooverdian/hh-school-git-learning.git'
...
$
```

Если сделать reset до тега 1.1 – это удалит последние два коммита ветки, которые были только что созданы

Теперь деревья коммитов локального и удалённого репозитория не совпадают и push заканчивается ошибкой

Use the Force Luke!

```
$ git push --force
Total 0 (delta 0), reused 0 (delta 0)
To github.com:gooverdian/hh-school-git-learning.git
 + bda60a6...a2f207f master -> master (forced update)
$
```

git push --force – независимо от состояния удалённого репозитория, он примет ту историю коммитов, которую передали вы, опасная штука

В ветках, в которых работают несколько разработчиков, её делать почти нельзя, потому что кто-то мог получить историю коммитов до удаления, или успеть закоммитить до пуша, это вызовет проблемы

Последствия

Теперь репозиторий на GitHub уверен, что никакого merge с веткой не было, этих коммитов не существует в истории

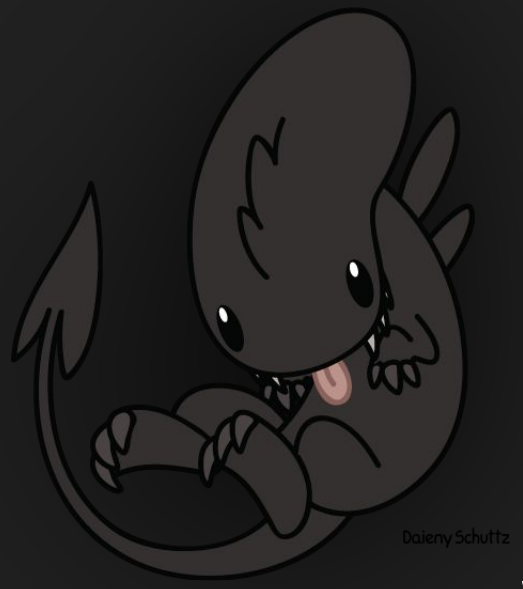
Остался pull request, но он теперь не отражает истинного положения вещей

Операции, меняющие историю нужно делать с чётким пониманием того, что, как и зачем, вы собираетесь делать

Если ошибётесь – можно подпортить жизнь себе и другим

Восстановить старую историю можно, если она есть у кого-нибудь в локальном репозитории, или в reflog

От создателей этой презентации
В главных ролях школьники
В роли злодея hh-school-git-crash-course



ЧУЖОЙ

Репозиторий

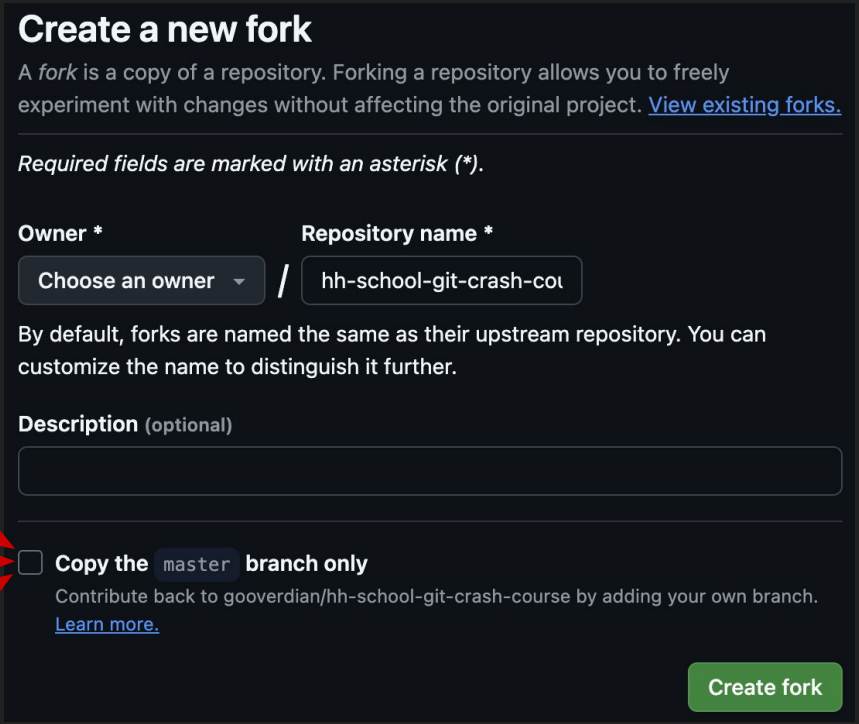
11 февраля 2026

А вот и он

https://github.com/gooverdian/hh-school-git-crash-course

Зайдите на страницу моего репозитория и в правой верхней части нажмите Fork.

Снимите галку "Copy the master branch only", вам понадобятся все ветки и теги.



Вилка линии судьбы

Fork создаёт у вас копию чужого репозитория, это копия теперь принадлежит вам, и у вас на неё полные права.

Пушить напрямую в чужой репозиторий невозможно без настройки прав мейнтейнером.

Сценарий обычно такой:

- Вы делаете fork, если уже есть – синхронизируете его с основным
- Разрабатываете в своём репозитории
- Делаете Pull Request в основной репозиторий из своего

Атака клонов

```
$ git clone https://github.com/<USERNAME>/hh-school-git-crash-course.git
Cloning into 'hh-school-git-crash-course'...
remote: Enumerating objects: 60, done.
remote: Counting objects: 100% (60/60), done.
remote: Compressing objects: 100% (28/28), done.
remote: Total 60 (delta 29), reused 55 (delta 24), pack-reused 0
Receiving objects: 100% (60/60), 7.13 KiB | 7.13 MiB/s, done.
Resolving deltas: 100% (29/29), done.
$
```

git clone – создаёт папку с копией удалённого репозитория, будет скопировано всё (коммиты, ветки, тэги, содержимое). Любая разработка в уже существующем репозитории начинается с клона.

Важно: клонируйте свой репозиторий, не мой! Иначе не сможете пушить

История одного репозитория

Есть файл sample.md, который был отредактирован в каждой ветке

Некоторые ветки репозитория были использованы в этой презентации, с помощью них можно повторить то, о чем я рассказывал (conflicting, rebase-me)

Ветки merge-me и make-me-friendly для домашнего задания

Финишная кривая

Домашнее задание – сделать все задачи репозитория, указанные в README.md

IDDQD

Doom 2

No.

Что ещё посмотреть

Крутое интерактивное демо с анимацией (очень рекомендую):
https://learngitbranching.js.org/

У Майкрософт есть неплохой бесплатный курс:
https://learn.microsoft.com/ru-ru/training/paths/github-foundations/

Книга про гит, написанная разработчиками гит, содержит вообще всё, но смысла читать её полностью особо нет, помогает ситуативно:
https://git-scm.com/book/en/v2 или https://git-scm.com/book/ru/v2

Что ещё изучить

Модели ветвления (git-flow, github-flow)

Работа с git в IDE, включая все команды и приёмы, которые мы рассмотрели

Процесс код-ревью (на что обращать и не обращать внимание при проведении или прохождении ревью, практики хватит в течение школы)

Thank you for participating in this Aperture Science computer-aided enrichment activity.

Все пулл реквесты просмотрю и откомментирую

В лекцию попало не всё из того, что полезно бы знать,
поэтому лекция окончена, а презентация продолжается ещё немного

Не отпускает?

Поехали дальше

Save Our Souls

Коммит существует, пока есть хоть одна ссылка на него (ветка, тэг, либо он родитель другого коммита, на который есть ссылка).
Если вы потеряли коммит, или десяток – у вас есть несколько вариантов:

- git reflog. Он временно хранит всё подряд, все изменения, даже удалённые, можно сделать checkout и создать ветку. Это восстановит ссылку на коммит, а значит и сам коммит
- Если эти коммиты были в удалённом репозитории, а вы не пушили – они всё ещё там
- Если запушили, то потерянные коммиты могут быть у вашего коллеги в локальной копии, пока он не сделал fetch

HTTPS vs SSH

Можно настроить ssh соединение с GitHub и не вводить каждый раз логин и пароль.

Нужно создать себе rsa ключ, если уже есть, то лучше создать новый, не удаляя старый:

```
$ ssh-keygen -t rsa -b 4096 -C "email@example.com"
```

и добавить его публичную часть (содержимое .pub) в настройки профиля на GitHub – раздел SSH and GPG keys, нажать New SSH key, указать название, вставить публичный ключ, нажать Add SSH key.

После этого, все настроенные ссылки на origin нужно переделать в SSH:

```
$ git remote set-url origin git@github.com:/<USERNAME>/<REPOSITORY>.git
```

Не только лишь всё

git хранит в репозитории не только содержимое файла, но и его модификаторы, mod (rrwwxx, или 4 цифры похожие на 0755)

Изменение mod считается изменением файла, его можно закоммитить

Используется, например, чтобы сохранить флаг execute на исполняемый файл и не договариваться о том, чтобы все делали chmod +x

Изменение модификатора лучше делать отдельным коммитом, чтобы было заметнее в истории

Локальный конфликт

Границы конфликта вычисляются git по затронутым строкам, если изменённые строки не соприкасаются (их номера различаются больше, чем на 2) – конфликта не будет

Это может выстрелить в ногу, но очень редко

Например, вы добавили вызов функции, а коллега независимо от вас изменил состав параметров в декларации этой функции, конфликта не будет, но и работать не будет

Rename ≠ Modify

Если файл был переименован, его историю всё равно можно проследить, но если одновременно с переименованием, файл изменён – git может решить, что старый файл удалён, а новый создан, история потеряется

Решение об этом принимается на основании того, сколько строк в файле было изменено, по умолчанию, граница – это 50% файла, если изменений меньше – переименован, если больше – это новый файл. Эту цифру можно менять

Чтобы не терять историю, переименовывать файлы лучше отдельными коммитами

И целого мира мало

git add . – добавляет все файлы в индекс, не нужно писать их имена

Полезно, если вы абсолютно уверены в правильно настроенном .gitignore, и в том, что не вносили изменений, которые не собираетесь коммитить

Может сыграть против вас, и это легко не заметить

Тоже самое касается git commit -a, но в меньшей степени, потому как он не добавляет неотслеживаемые файлы

Всадник без головы

Как я уже говорил, указатель HEAD можно перемещать вручную, например, сделав git checkout <COMMIT_HASH>

Состояние, когда HEAD не привязан ни к какой ветке – 'detached HEAD state'

Вы получите рабочую копию коммита. Полезно, когда нужно глянуть, как было в определенный момент. Можно вносить изменения и коммитить их, но они потеряются, если переключиться на любую ветку

Чтобы не потерялись – можно создать ветку прямо из detached HEAD state

Чтобы просто выйти из detached HEAD state нужно переключиться на ветку

Мелочь, а приятно

При отсутствии пустой строки в конце файла, GitHub нарисует символ ⊖

Пустые строки в конце файла нужны по нескольким причинам:

- Если вы добавили одну строку в файл, в котором не было пустой строки в конце, git diff покажет, что вы изменили две строки – это может привести к конфликту там, где его не должно было быть
- При простой склейке двух файлов, отсутствие пустой строки в конце первого, склеит вместе последнюю строку первого и первую строку второго
- Некоторые приложения игнорируют строки файлов без переноса в конце

На перехват!

В действия, производимые git, можно вклиниваться, используя перехватчики. Они пригодятся для проверки кода, автоматизации, и отмены действий при определённых условиях

Например, перехватчик pre-commit выполняется первым сразу после вызова git commit, его можно использовать, чтобы расставить везде пустые строки в конце, убрать лишние пробелы на концах строк, или проверить стиль кода. В случае, если перехватчик завершится с ненулевым кодом – коммит будет отменён.

Перехватчик – это исполняемый скрипт. Подробнее про перехватчики

Оружие массового поражения

Что если у вас уже есть репозиторий, в котором под 50 000 коммитов, и вам нужно изменить что-то одно в каждом из них? Например, сведения о лицензии, или удалить случайно забытый в самом начале пароль...

Можно попробовать rebase -i, но это мучение, есть вариант получше

git filter-branch – команда позволяет что-то сделать с каждым из коммитов ветки, начиная с первого коммита всего репозитория. Конечно же, полностью меняя хэши всех коммитов

Умеет изменять содержимое или описание и авторство коммитов и выделять отдельный репозиторий из поддиректории текущего. Подробности тут

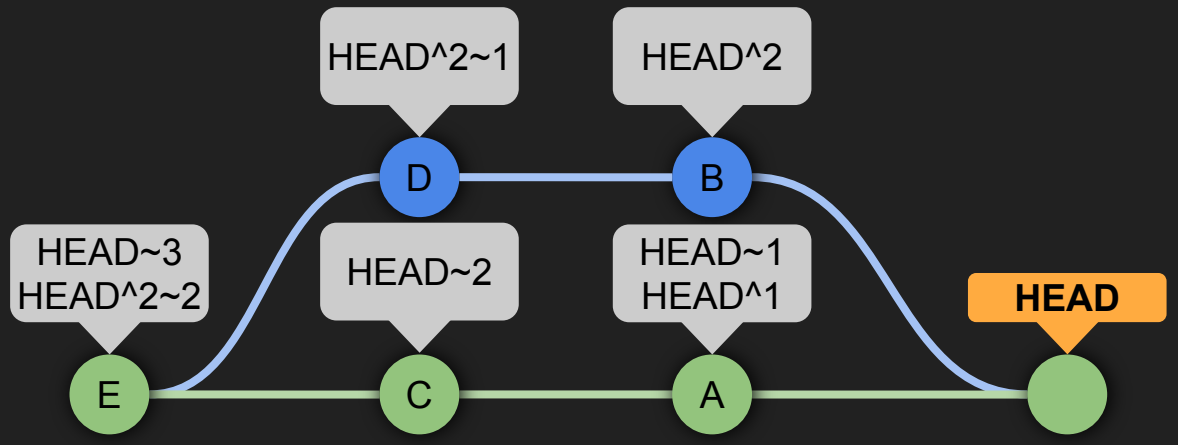
Ссылки в git: ~ и ^

Ссылаются на родительский коммит

В случае, если текущий коммит merge commit – будут указывать на коммит ветки, в которую были влиты изменения

После символа можно добавить число, которое и определяет различия между этими символами. Если число не указано, будет взято 1.

Для ^ число определяет номер родительского коммита в слиянии

Для ~ число определяет на сколько коммитов вверх нужно перейти, для слияний всегда выбирается ветка, в которую были влиты изменения

Ссылки в git: пример